Customer Segmentation for an Online Retailer: K-Means versus DBSCAN

Essentials of Machine Learning : Assignment 2 (Individual)

Name : S Dinath

Registration Number : E2340013

University of Moratuwa - BIT

Abstract

This report segments 4,334 customers of a UK online retailer using four behavioural features (Recency, Frequency, Monetary value and product variety) derived from 541,909 transaction lines. K-Means and DBSCAN are trained on the same log-transformed, standardised feature space and compared on the Silhouette, Davies-Bouldin and Calinski-Harabasz metrics, runtime, stability and noise handling. K-Means with K = 4 produces four interpretable segments, in which the top 18.9% of customers generate 68.0% of revenue. DBSCAN (eps = 0.5, min_samples = 8) labels 8.47% of customers as noise, and those customers account for 46.0% of revenue. K-Means is recommended as the main segmentation method, with DBSCAN used to flag extreme accounts for personal attention.

1. Dataset Exploration (Task 1)

1.1 Dataset and clustering objective

The Online Retail dataset (Chen, 2015) from the UCI Machine Learning Repository contains the transactions of a UK-based online retailer between 01/12/2010 and 09/12/2011. It has **541,909 records and 8 variables**. The objective is unsupervised: group customers by purchasing behaviour so that each group can be targeted differently. Because the data is at transaction level, customer-level features must be engineered before clustering (Section 2).

Variable	Type	Use in this study
InvoiceNo	String (prefix C = cancellation)	Frequency (distinct invoices); cancellation filter
StockCode	String	UniqueProducts (distinct codes); non-product filter
Description	String	Excluded (free text, duplicates StockCode information)
Quantity	Integer	Monetary (Quantity × UnitPrice)
InvoiceDate	Datetime	Recency
UnitPrice	Float	Monetary
CustomerID	Float (contains missing values)	Aggregation key
Country	String	Excluded (see Section 2.2)

The **relevant clustering features** are four customer-level variables: Recency, Frequency, Monetary and UniqueProducts. The dataset has 4372 customers before cleaning and 4,334 after cleaning.

1.2 Data quality inspection

- **Missing values:** CustomerID is missing in 135,080 rows (24.9%). Description has 1454 missing values.
- **Duplicates:** 5268 exact duplicate rows exist in the raw data. After rows without a CustomerID were removed, 5,225 duplicates remained and were dropped.
- **Cancellations:** invoices starting with C carry negative quantities.There are 9,288 such lines in the raw dataset; 8,872 such lines were removed during cleaning.
- **Invalid values:** Quantity or UnitPrice of zero or below occurs in 10,624 and 2,517 rows respectively; a further 40 rows remained after cancellations were removed.
- **Non-product codes:** postage, bank charges and manual adjustments share the StockCode column with real products; 1,542 such lines were removed.

	missing	pct
InvoiceNo	0	0.00
StockCode	0	0.00
Description	1454	0.27
Quantity	0	0.00
InvoiceDate	0	0.00
UnitPrice	0	0.00
CustomerID	135080	24.93
Country	0	0.00

```
Exact duplicate rows: 5268
Cancelled invoices (InvoiceNo starts with 'C'): 9288
Rows with Quantity <= 0 : 10624
Rows with UnitPrice <= 0: 2517
```

	count	mean	std	min	25%	50%	75%	max
Quantity	541,909.00	9.55	218.08	-80,995.00	1.00	3.00	10.00	80,995.00
UnitPrice	541,909.00	4.61	96.76	-11,062.06	1.25	2.08	4.13	38,970.00

	count	unique	top	freq
InvoiceNo	541909	25900	573585	1114
StockCode	541909	4070	85123A	2313
Description	540455	4223	WHITE HANGING HEART T-LIGHT HOLDER	2369
Country	541909	38	United Kingdom	495478

Table 1: Descriptive statistics of the raw transaction data.

1.3 Visualizations and key observations

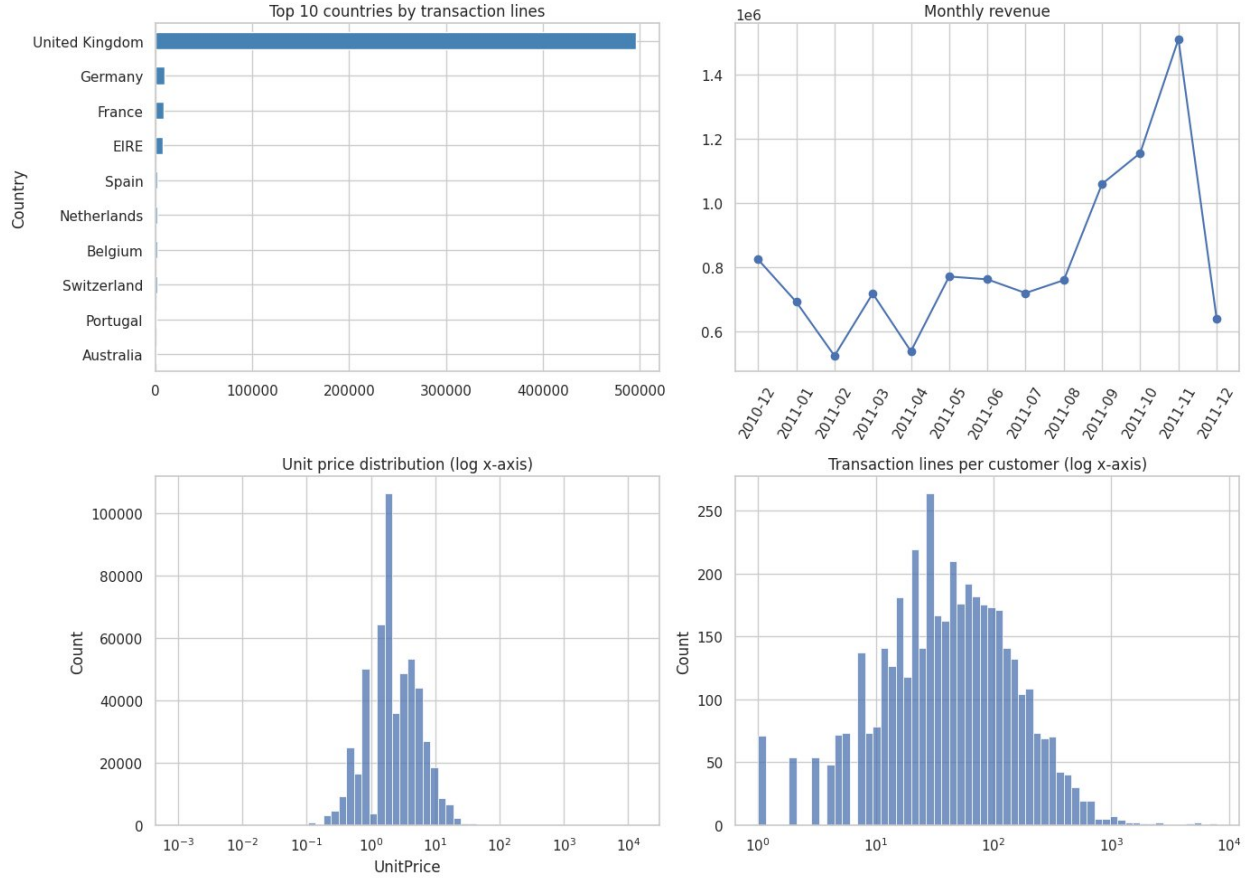


Figure 1: Exploratory plots of the cleaned positive-value transactions.

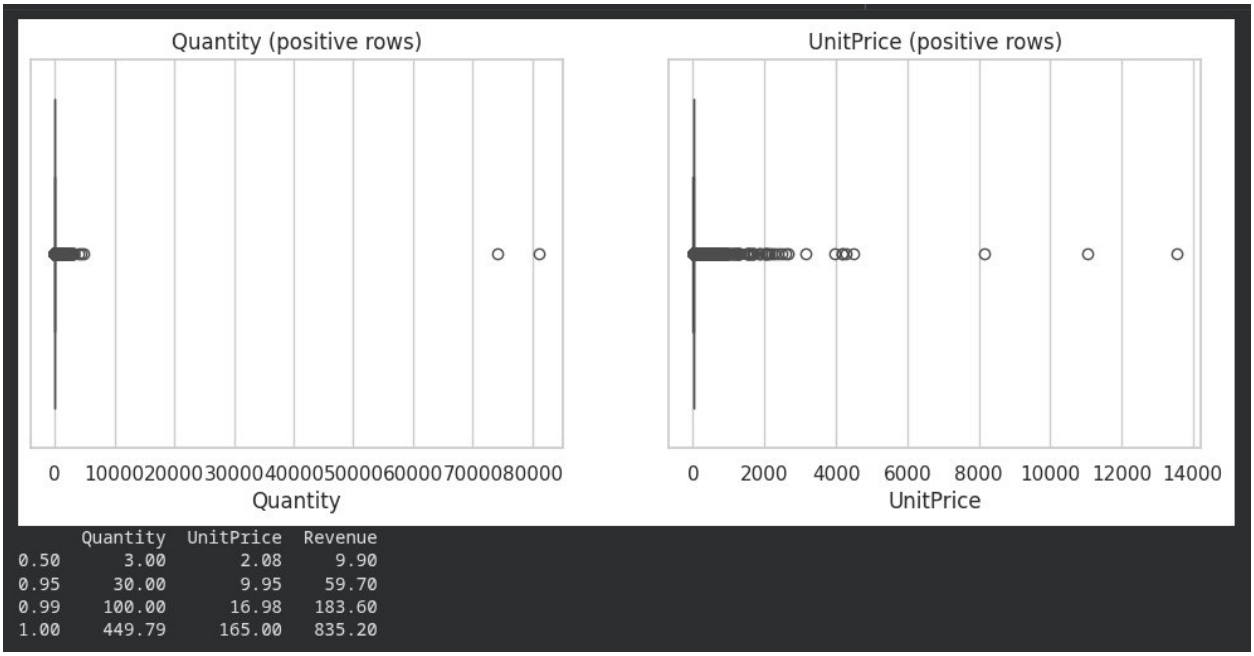


Figure 2: Boxplots showing extreme Quantity and UnitPrice values.

Key observations:

1. One quarter of rows have no CustomerID, so they cannot be assigned to a customer and must be removed.
2. Transaction values are heavy-tailed. Unit price and lines per customer only become readable on log axes (Figure 1, panels 3 and 4).
3. Cancellations, zero or negative values and non-product codes would distort spend if left in the data.
4. The United Kingdom heavily dominates the transaction volume, representing the overwhelming majority of transaction lines compared to all other international markets.
5. Monthly revenue shows a steady rise over the course of the year, culminating in a significant sales peak in November 2011, followed by a sharp drop-off in early December.
6. At customer level the features are strongly right-skewed (Section 2.3), which motivates a transformation before distance-based clustering.

2. Preprocessing (Task 2)

2.1 Cleaning

Step	Rows remaining	Rows removed
Raw data	541,909	-
Drop missing CustomerID	406,829	135,080
Drop exact duplicate rows	401,604	5,225
Drop cancelled invoices	392,732	8,872
Drop Quantity ≤ 0 or UnitPrice ≤ 0	392,692	40
Drop non-product StockCodes	391,150	1,542

Table 2: Cleaning pipeline. In total 150,759 lines (27.8%) were removed.

Rows without a CustomerID cannot contribute to customer-level features. Duplicates would double-count purchases. Cancelled invoices and non-positive values were excluded so that spend reflects genuine purchases; this means return behaviour is not modelled (see Limitations). Only StockCodes beginning with five digits were kept, which retains real products and drops postage, fees and manual entries.

2.2 Feature engineering and exclusions

A snapshot date (last invoice date plus one day) was used to compute recency. Each customer is described by:

Feature	Definition
Recency	Days between the snapshot date and the customer's last purchase
Frequency	Number of distinct invoices
Monetary	Total spend, the sum of Quantity × UnitPrice
UniqueProducts	Number of distinct StockCodes purchased

RFM is a standard basis for customer segmentation (Chen et al., 2012), and UniqueProducts adds a product-variety dimension. Country was excluded because the customer base is dominated by one country and a categorical variable would need encoding that distorts Euclidean distances. Description was excluded as free text. Average basket value was not added because it is a ratio of Monetary to Frequency. All four retained features are numeric, so **no categorical encoding was needed**. The result is 4,334 customers.

Feature	Mean	Std	Min	25%	Median	75%	Max
Recency	92.70	100.18	1	18	51	143	374
Frequency	4.25	7.63	1	1	2	5	206
Monetary	2,015.97	8,903.67	3.75	304.24	662.57	1,631.62	279,138.02
UniqueProducts	61.43	85.31	1	16	35	77	1,785

Table 3: Descriptive statistics of the customer-level features (n = 4,334).

2.3 Skewness, outliers and scaling

Feature	Skewness before	Skewness after log1p
Recency	1.24	−0.38
Frequency	11.95	1.21
Monetary	19.58	0.40
UniqueProducts	6.92	−0.24

Table 4: Skewness before and after the log(1 + x) transformation.

A log(1 + x) transformation was applied to all four features. It removed most of the skew in Monetary and UniqueProducts. Frequency remains moderately skewed (1.21) because about a third of customers have a single invoice, which creates a discrete mass at the minimum that no monotone transform removes.

Outliers were **kept rather than deleted**. The largest customers (maximum spend £279,138) are plausibly genuine high-value accounts, and removing them would erase the most commercially important segment. The log transform limits their leverage on distance calculations, and DBSCAN is designed to flag extreme points as noise. Features were then standardised with StandardScaler (mean 0, standard deviation 1), because both algorithms are distance-based and the raw scales differ by orders of magnitude. The final matrix is 4,334 × 4.

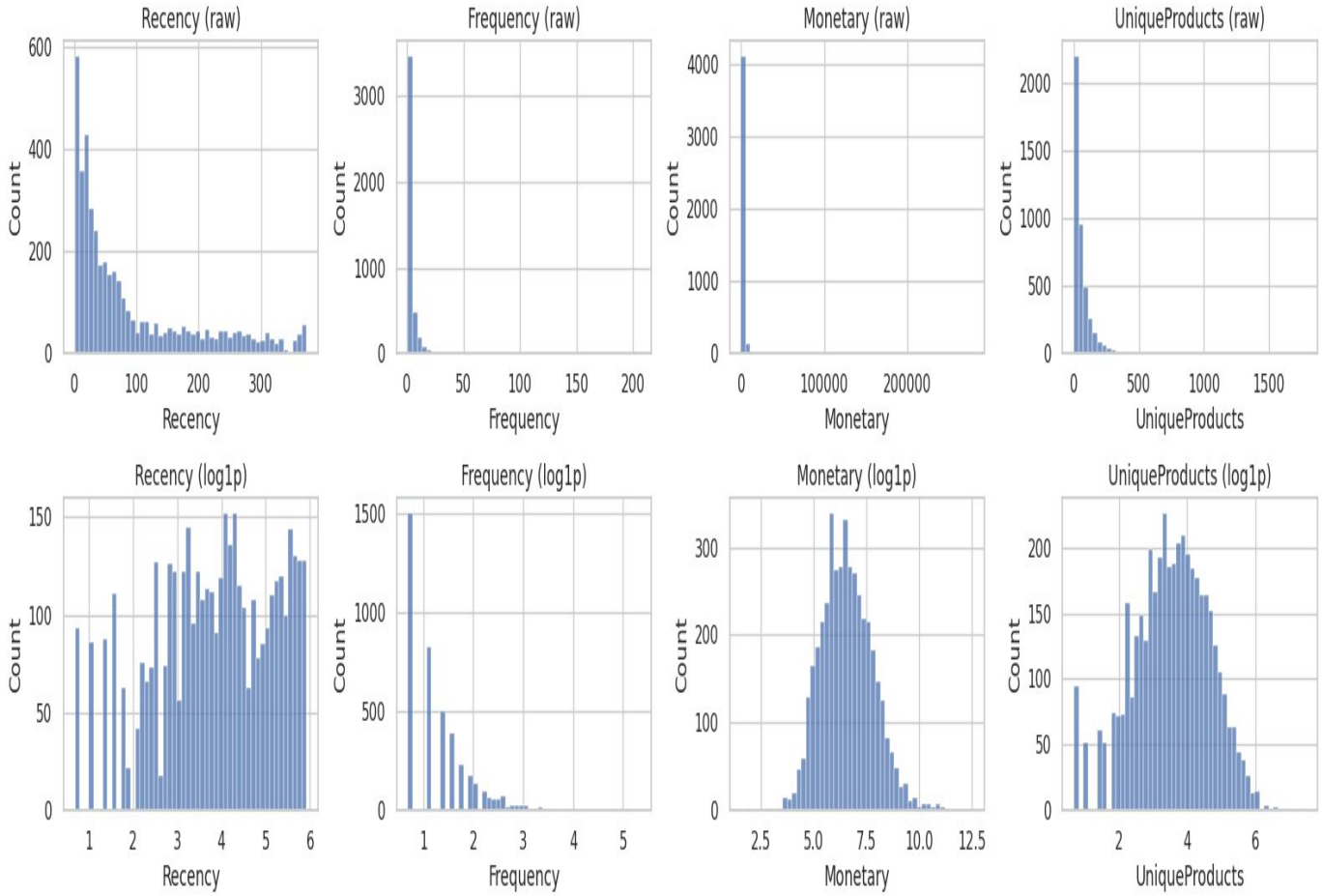


Figure 3: Feature distributions before (top) and after (bottom) the log transformation.

3. K-Means Clustering (Task 3)

K-Means (n_init = 10, random_state = 42) was trained for K = 2 to 10, which is nine candidate values.

K	Inertia	Silhouette	Davies–Bouldin	Calinski–Harabasz
2	9,125.82	0.40	0.95	3,897.36
3	7,083.82	0.30	1.14	3,134.09
4	6,141.43	0.28	1.20	2,630.95
5	5,298.98	0.25	1.19	2,458.42
6	4,810.49	0.23	1.23	2,253.88
7	4,413.39	0.23	1.17	2,111.64
8	4,107.29	0.22	1.20	1,990.50
9	3,858.62	0.23	1.16	1,888.30
10	3,641.09	0.23	1.17	1,807.08

Table 5: K-Means scan over candidate K.

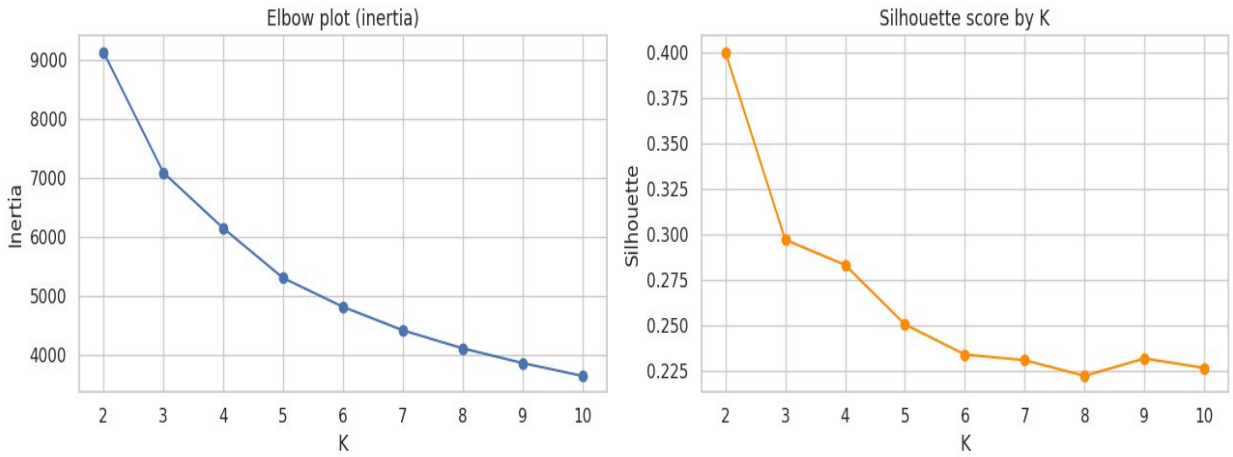


Figure 4: Elbow plot (left) and Silhouette score by K (right).

3.1 Choice of K

The Silhouette score is highest at K = 2 (0.40), but a two-group solution only separates active from inactive customers and is too coarse to act on. The elbow is gradual: inertia falls by about 2,042 from K = 2 to 3, by 942 from K = 3 to 4, by 842 from 4 to 5 and by 488 from 5 to 6, so the largest bend sits at K = 3 to 4 and the gains diminish afterwards. **K = 4 was chosen** as a balance between the elbow, a Silhouette that is still the third highest (0.28) and interpretability: it yields four balanced, behaviourally distinct segments (821, 1,245, 1,448 and 820 customers). K = 3 (Silhouette 0.30) is a reasonable alternative. A Silhouette of 0.28 indicates weak structure, which is expected because RFM behaviour varies continuously rather than forming isolated groups.

3.2 Final clustering and profiles

The final model (K = 4) has Silhouette 0.283, Davies-Bouldin 1.203 and Calinski-Harabasz 2,631, with a runtime of 0.176 s. The two-dimensional PCA projection explains 85.9% of the variance.

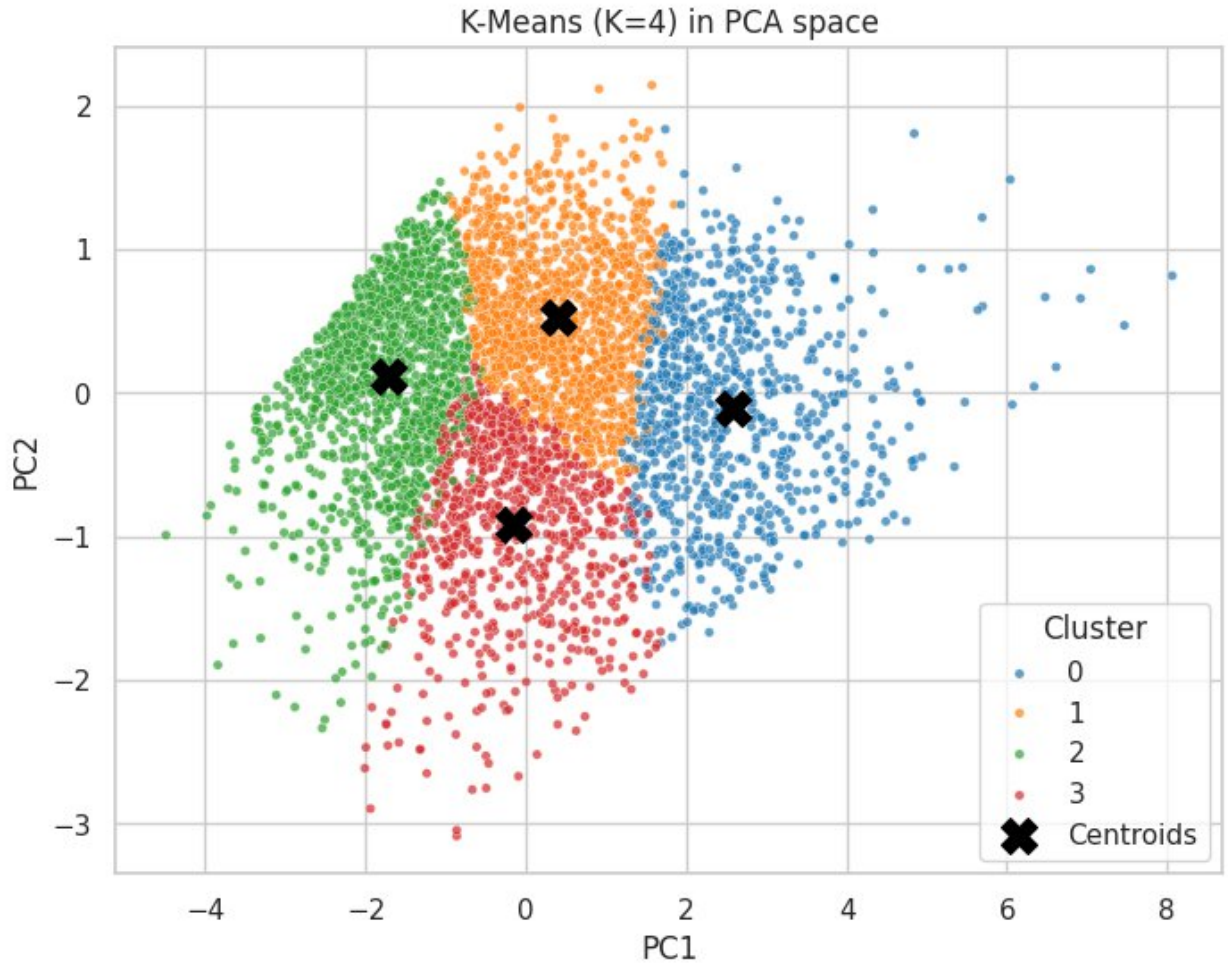


Figure 5: K-Means (K = 4) clusters projected onto the first two principal components.

Cluster	Customers	% of customers	Recency (days)	Frequency	Monetary (£)	UniqueProducts	% of revenue
0	821	18.9	14.3	12.6	7,236.7	155.7	68.0
1	1,245	28.7	85.8	3.5	1,507.1	70.5	21.5
2	1,448	33.4	184.3	1.3	304.3	15.7	5.0
3	820	18.9	20.0	2.2	584.0	34.1	5.5

Table 6: K-Means cluster profiles using the original (untransformed) features; values are cluster means.

Cluster	Recency	Frequency	Monetary (£)	UniqueProducts
0	10.0	9.0	3,353.0	119.0
1	65.0	3.0	1,096.3	58.0
2	183.5	1.0	250.6	13.0
3	17.5	2.0	464.0	28.0

Table 7: Cluster medians. Means exceed medians in the spending columns because of right skew inside each cluster.

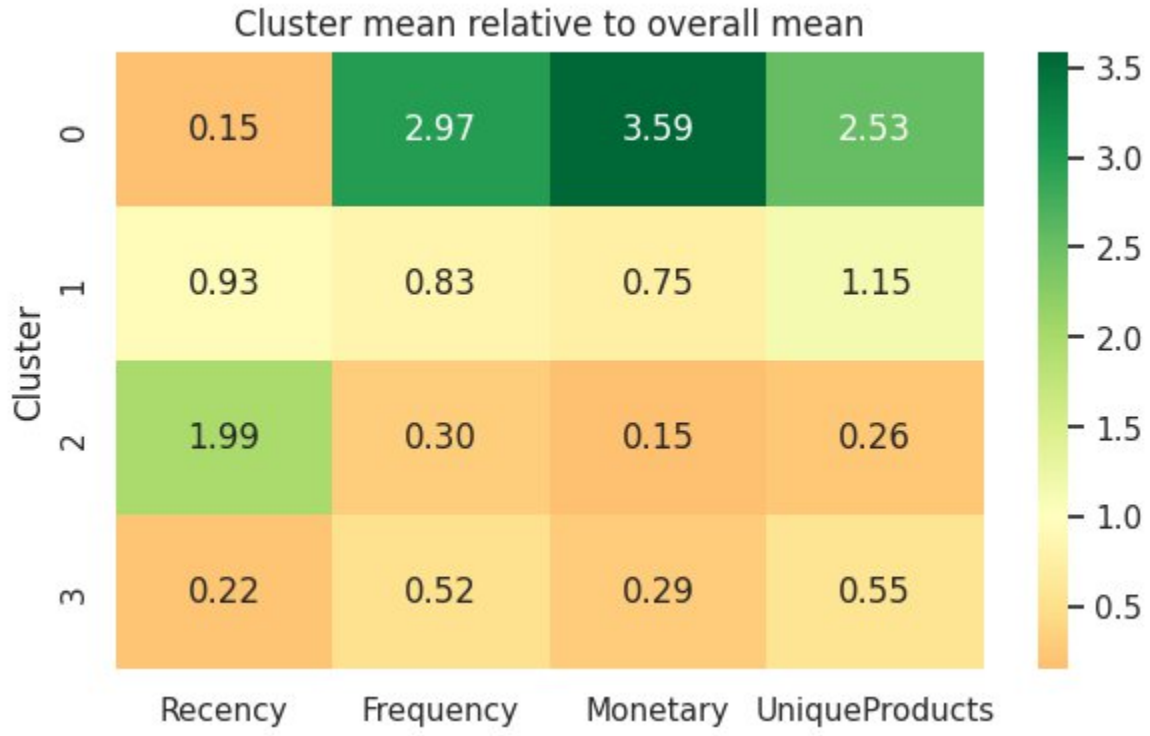


Figure 6: Each cluster's mean as a multiple of the overall mean (1.0 = average).

4. DBSCAN Clustering (Task 4)

4.1 Parameter selection

min_samples was set to 8, following the common heuristic of at least twice the number of dimensions (2 × 4). **eps** was investigated with a k-nearest-neighbour distance plot and a parameter grid. In the k-distance plot the curve rises gently through roughly 0.3 to 0.7 and turns sharply upward from about 0.8 to 1.0 for min_samples = 8.

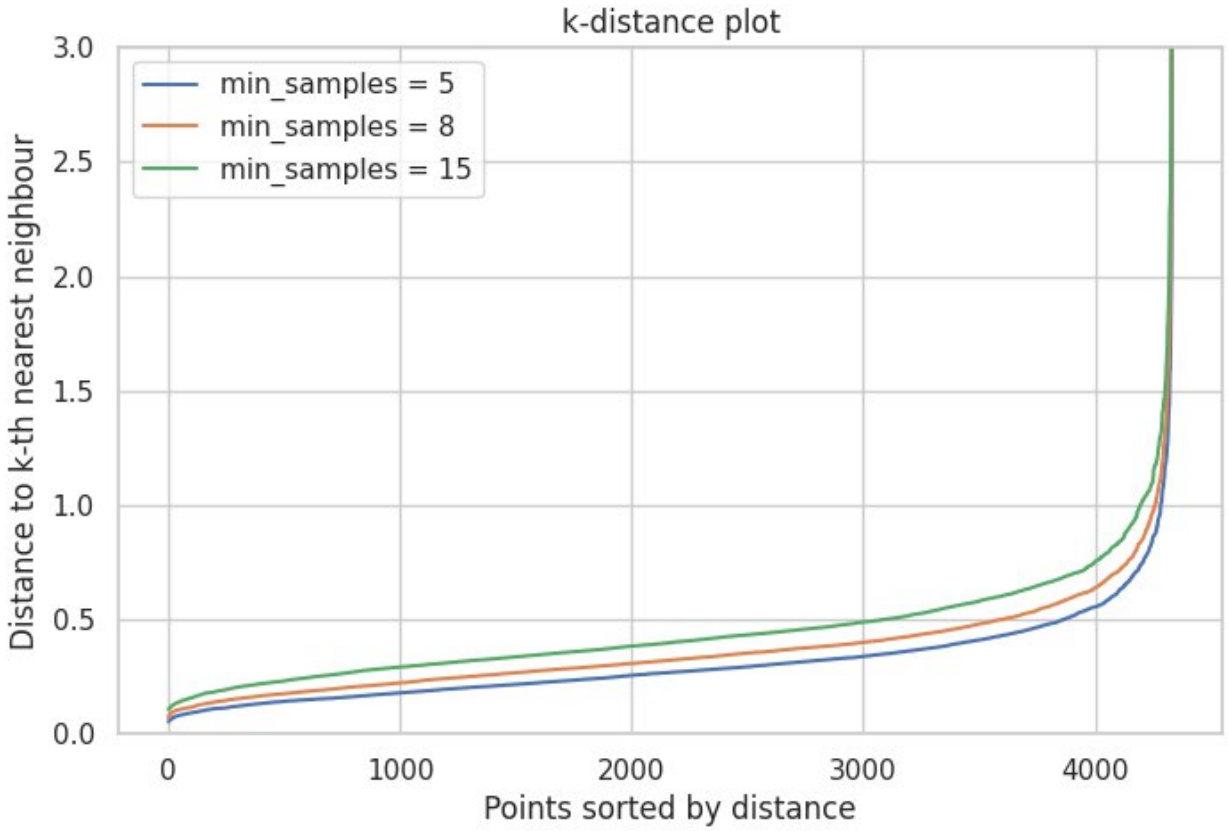


Figure 7: k-distance plot used to select eps.

eps	Clusters	Noise (%)	Silhouette (non-noise)
0.2	22	73.23	−0.12
0.3	16	41.26	−0.06
0.4	4	18.50	0.11
0.5	**4**	**8.47**	**0.16**
0.6	2	4.50	0.18

Table 8: DBSCAN grid for min_samples = 8 (full grid in Appendix A5).

The grid shows that small eps fragments the data into many clusters with most points labelled noise, while larger eps merges everything. With min_samples = 5 the cluster count runs 50, 29, 11, 4, 1, 2, 1, 1, 2 as eps increases from 0.2 to 1.0, so the knee region (0.8 to 1.0) collapses to one or two clusters. **eps = 0.5 was chosen** as the largest value that keeps more than one cluster while holding noise under 10%.

4.2 Results

DBSCAN (eps = 0.5, min_samples = 8) found **4 clusters** of 2,519, 1,432, 9 and 7 customers. **367 customers (8.47%) are noise**, so 3,967 customers (91.53%) are retained for metric computation. Runtime was 0.138 s.

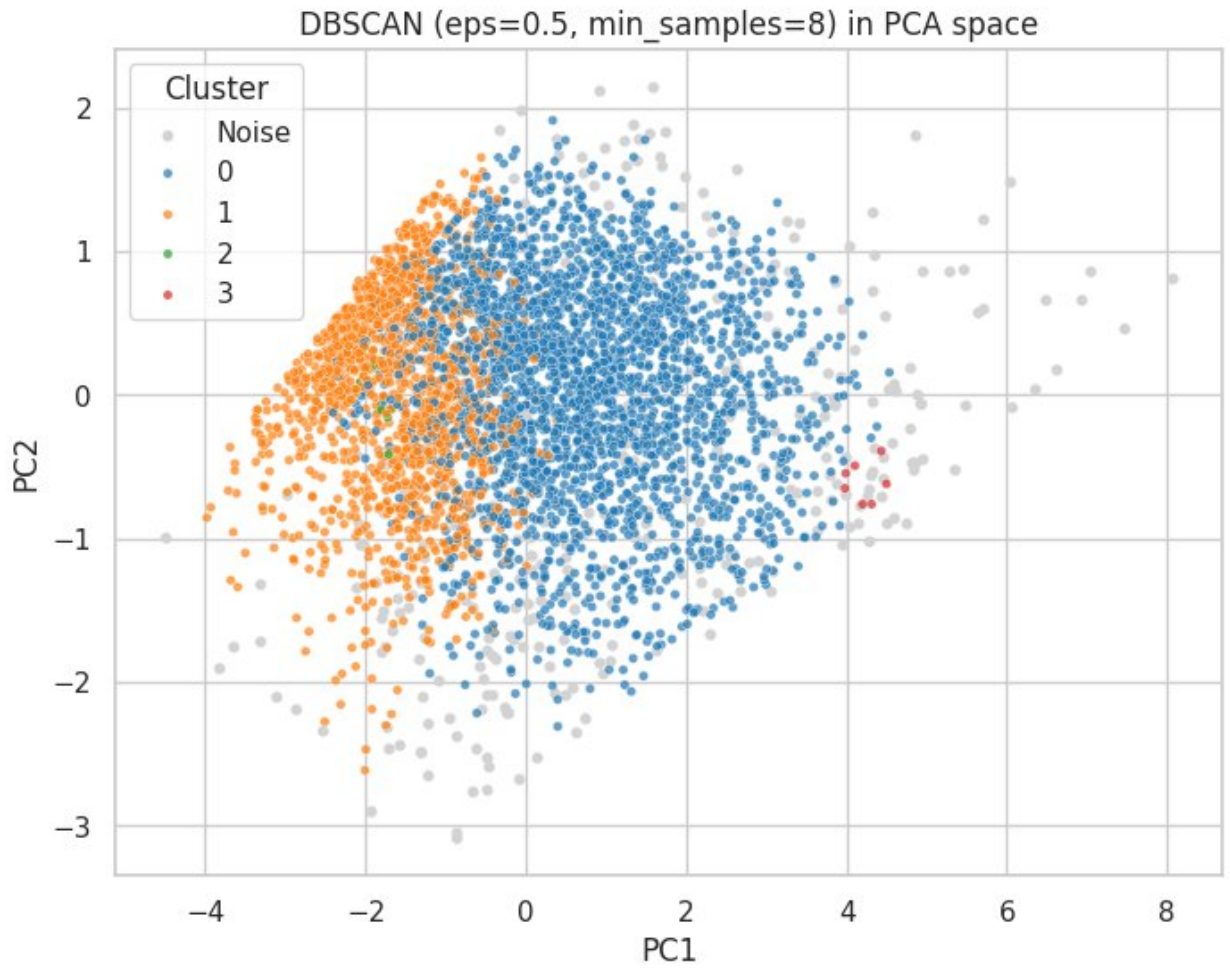


Figure 8: DBSCAN clusters and noise projected onto the first two principal components.

Group	Customers	Recency	Frequency	Monetary (£)	UniqueProducts
Noise (−1)	367	49.3	12.8	10,950.1	104.5
0	2,519	60.0	4.8	1,653.3	78.0
1	1,432	161.1	1.0	327.0	21.0
2	9	210.4	2.1	736.9	1.2
3	7	1.6	23.9	11,287.9	160.0

Table 9: DBSCAN group means on the original features.

In practice DBSCAN separates two broad groups: active customers (cluster 0) and one-time lapsed buyers (cluster 1, every customer with exactly one invoice on average). Clusters 2 and 3 are micro-clusters: cluster 2 buys about one distinct product, and cluster 3 holds seven extreme high-frequency, high-spend accounts.

5. Comparison of Algorithms (Task 5)

Measure	K-Means	DBSCAN
Selected parameters	K = 4, n_init = 10	eps = 0.5, min_samples = 8
Number of clusters	4	4 (plus noise)
Cluster sizes	821, 1,245, 1,448, 820	2,519, 1,432, 9, 7 (noise: 367)
Silhouette Score	0.283	0.164
Davies-Bouldin Index	1.203	0.841
Calinski-Harabasz Index	2,630.9	820.4
Noise points (%)	0.00	8.47
Runtime (seconds)	0.176	0.138

Table 10: Required performance comparison. All runs use the same standardised 4,334 × 4 feature space. K-Means metrics use all 4,334 customers. ***DBSCAN metrics exclude the 367 noise points and use only the 3,967 retained customers (91.53%)****, so the two columns are not computed on the same sample. Runtimes are single runs on the same machine.*

Metric	K-Means (all 4,334)	K-Means (same 3,967 points as DBSCAN)	DBSCAN (3,967 non-noise points)
Silhouette	0.283	0.301	0.164
Davies-Bouldin	1.203	1.168	0.841
Calinski-Harabasz	2,630.9	2699.43	820.4
% of records used	100%	91.53%	91.53%

Table 11: Like-for-like comparison on identical records. Compare the last two columns directly.\



Figure 9: K-Means (left) and DBSCAN (right) in the same PCA space.

Cluster shape and separation. K-Means partitions the space into compact, roughly convex regions around centroids and assigns every customer to one of them. DBSCAN follows density: it finds one large dense region, a second region of one-time buyers and a few tiny groups. The metrics disagree. DBSCAN is better on Davies–Bouldin (0.841 against 1.203) but worse on Silhouette (0.164 against 0.283) and Calinski-Harabasz (820 against 2,631). Two of DBSCAN's four clusters contain only 9 and 7 customers, so its cluster count overstates how much structure it found. Scores computed on different samples should not be read as a direct ranking; Table 11 gives the like-for-like view.

Stability. On 80% subsamples, DBSCAN reproduced its full-data partition with an Adjusted Rand Index (ARI) of 0.951 +- 0.006, while K-Means achieved 0.622 +- 0.301. The large K-Means spread shows that, because the data is a continuum, removing random boundary customers can shift where the centroids settle. DBSCAN's high ARI should be read with care: its output is dominated by two large clusters and a noise group, so the partition is coarse and easy to reproduce. DBSCAN is stable under resampling but **sensitive to parameters**: raising eps from 0.5 to 0.6 collapses the result from 4 clusters to 2.

Noise handling. DBSCAN labels 367 customers (8.47%) as noise. These customers average 12.8 invoices and £10,950 in spend, and **together they account for 46.0% of total revenue**. Treating the most valuable customers as noise is a serious drawback for segmentation, although it makes DBSCAN a useful outlier detector.

Runtime. Both algorithms finish in well under a second (0.176 s and 0.138 s), so runtime does not separate them at this scale.

Recommendation. K-Means with K = 4 is recommended for customer segmentation because every customer receives an actionable segment and the clusters are balanced and interpretable. DBSCAN should be used alongside it to isolate extreme accounts for a dedicated high-touch track.

6. Interpretation of Results (Task 6)

Cluster	Name	Customers	Share of revenue	Description
0	**Champions**	821 (18.9%)	68.0%	Bought a median of 10 days ago, about 9 invoices (mean 12.6), mean spend £7,237 and about 156 distinct products. Frequent, recent, high-value buyers, probably including trade or wholesale accounts.
1	**Occasional Regulars**	1,245 (28.7%)	21.5%	Median 65 days since last purchase (mean 86), about 3 invoices and median spend £1,096. Reliable but infrequent buyers who form the steady middle of the base.
2	**Lapsed One-Timers**	1,448 (33.4%)	5.0%	About 184 days since last purchase, a median of 1 invoice and median spend £251. One-off or long-lapsed customers.
3	**Recent Low-Spenders**	820 (18.9%)	5.5%	Bought recently (median 17.5 days) but only about 2 invoices, median spend £464 and 28 products. Given that Cell 24 shows a very short median tenure of 34.0 days for this group (Cluster 2), this confirms they are recently acquired, so we rename this group to 'New Customers'.

Table 12: Named K-Means segments, supported by Tables 6 and 7.

6.1 Actionable recommendations

1. **Protect Champions with a VIP programme.** This group is 18.9% of customers but 68.0% of revenue, so losing even a few is costly. Offer dedicated account management, volume-based pricing and early access to stock, and place the DBSCAN noise accounts (46.0% of revenue) on a personal-contact watch-list.
2. **Convert Recent Low-Spenders into repeat buyers.** They have bought recently but with little variety (28 products against 119 for Champions). Send an automated post-purchase email series with popular-product recommendations and bundle offers, and track the share that move into the Occasional Regulars or Champions segments.
3. **Re-engage Occasional Regulars before they lapse.** Their median recency is 65 days, while lapsed customers sit at about 184 days. Trigger a reminder or modest incentive at roughly 60 to 90 days since the last order.
4. **Limit spend on Lapsed One-Timers.** They are a third of customers but 5.0% of revenue. Run a low-cost win-back campaign as an A/B test and stop if the response rate is poor.

6.2 Limitations

- **Outliers:** Log transformation reduces but does not remove the influence of very large customers. K-Means pulls its Champions centroid towards them, while DBSCAN discards them as noise.
- **Feature selection:** Only four behavioural features were used. Product category, return behaviour, country, seasonality and customer tenure are not captured, and Frequency and UniqueProducts may partly overlap (see Appendix A2). Frequency stayed right-skewed after transformation.
- **Parameter sensitivity:** K-Means requires K in advance, and the Silhouette score favoured K = 2 while K = 4 was chosen for interpretability. DBSCAN's cluster count ranged from 50 to 1 across the eps grid, so its result depends heavily on eps.
- **Data scope:** About a quarter of rows had no CustomerID and were dropped, and the data covers a single year from one retailer. The two-dimensional PCA plots lose some variance, so overlap in the plots is not a full picture.

7. Conclusion (Task 7)

Cleaning the Online Retail data and aggregating it into Recency, Frequency, Monetary and UniqueProducts features gave 4,334 customers. Customer behaviour forms a continuum with weak-to-moderate cluster structure (K-Means Silhouette 0.283). K-Means with K = 4 produced four interpretable segments, led by Champions, who are 18.9% of customers and generate 68.0% of revenue. DBSCAN (eps = 0.5, min_samples = 8) found two large groups, two micro-clusters and 8.47% noise; the noise was mostly very high-value customers (46.0% of revenue), and the result was sensitive to eps. **K-Means is the preferred method** for segmentation, with DBSCAN retained as an outlier-flagging tool.

Possible improvements include adding tenure, return rate, basket size and product-category features (for example from text analysis of Description), trying Gaussian mixture models, hierarchical clustering or HDBSCAN (which removes the single-eps limitation), and validating the segments on a later time period.

Generative AI Declaration

I used generative AI assistants Claude by Anthropic to help structure the notebook and compare between Jupiter Notebook vs Google colab, draft code for cleaning, feature engineering, K-Means, DBSCAN and metric computation, debug errors and draft wording for the report. I ran all code myself, checked the outputs against my own results and edited the interpretations and recommendations.

References

Caliński, T. and Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3(1), 1–27.

Chen, D. (2015). *Online Retail* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5BW33

Chen, D., Sain, S. L. and Guo, K. (2012). Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3), 197–208.

Davies, D. L. and Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.

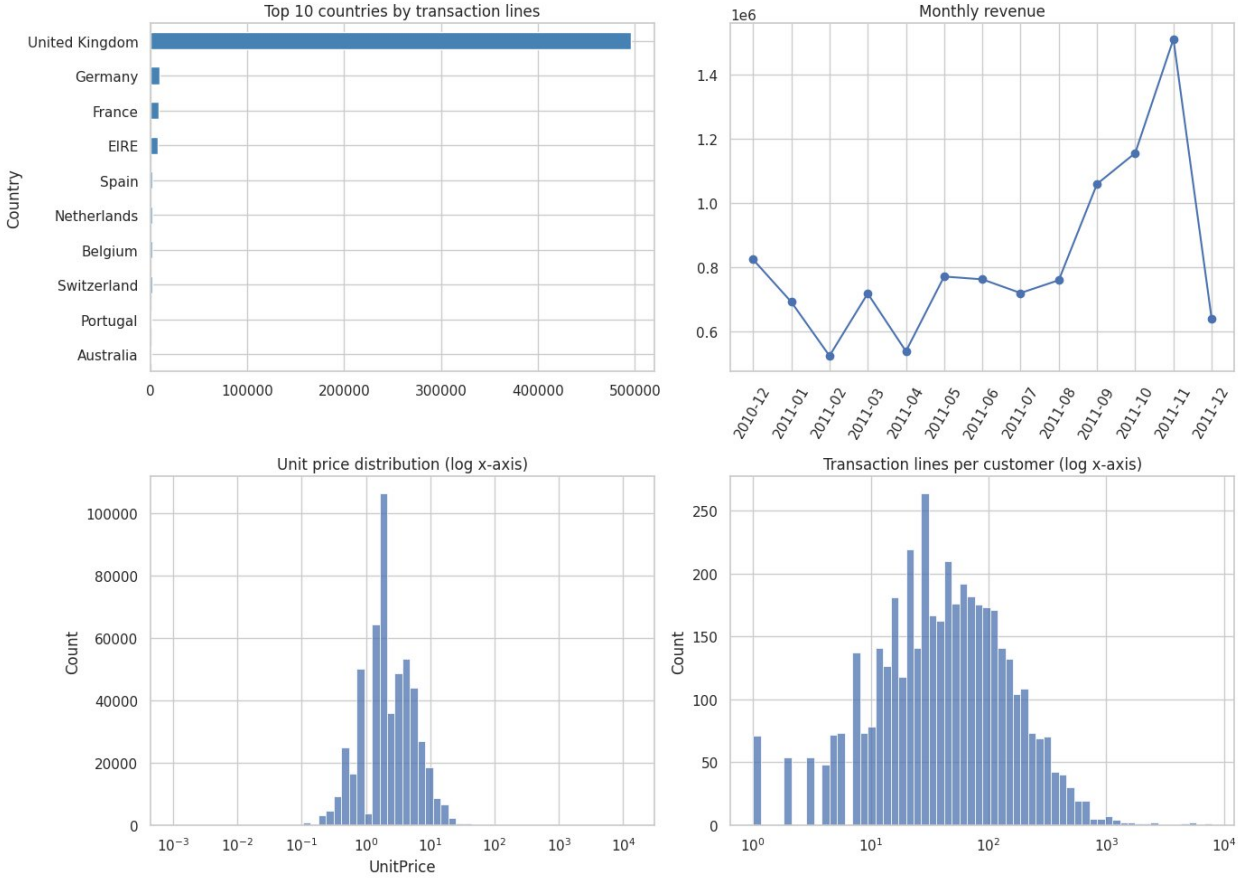
Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD-96*, 226–231.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

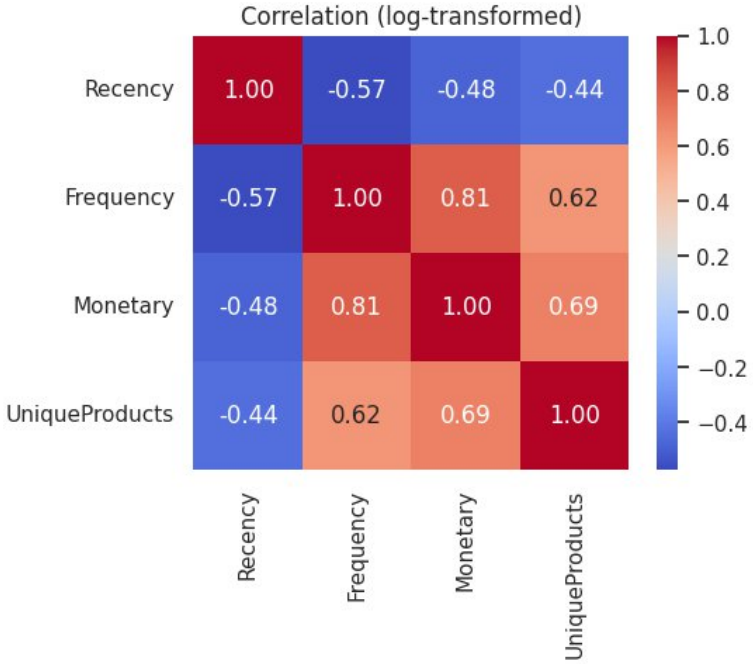
Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

Appendix

A1. Outlier boxplots.



A2. Correlation heatmap of the log-transformed features.



A3. Full DBSCAN parameter grid.

```
grid = []
for ms in [5, 8, 10, 15]:
    for eps in np.round(np.arange(0.2, 1.05, 0.1), 2):
        lab = DBSCAN(eps=eps, min_samples=ms).fit_predict(X)
        mask = lab != -1
        n_cl = len(set(lab[mask]))
        grid.append({
            "eps": eps, "min_samples": ms, "clusters": n_cl,
            "noise_%": (~mask).mean() * 100,
            "silhouette (non-noise)": silhouette_score(X[mask], lab[mask]) if n_cl >= 2 else np.nan,
        })
grid_df = pd.DataFrame(grid)
display(grid_df.round(3).to_string())
```

```
'    eps  min_samples  clusters  noise_%  silhouette (non-noise)\n0  0.20           5        50    60.64                  -0.19\n1  0.30           5        29    28.57                  -0.18\n2  0.40
5        11    13.13                  -0.04\n3  0.50           5         4     5.98                   0.18\n4  0.60           5         1     3.37                   NaN\n5  0.70           5
2     2.01               0.19\n6  0.80           5         1     1.36                 NaN\n7  0.90           5         1     0.88                 NaN\n8  1.00           5         2     0.58
0.55\n9  0.20           8        22    73.23                  -0.12\n10 0.30           8        16    41.26                  -0.06\n11 0.40           8         4    18.50                   0.11\n12
0.50           8         4     8.47                   0.16\n13 0.60           8         2     4.50                   0.18\n14 0.70           8     ...'
```

A4. Stability and sensitivity results.

```
K-Means ARI across seeds      : 0.920 (min 0.866)
K-Means ARI on 80% subsamples : 0.622 ± 0.301
DBSCAN  ARI on 80% subsamples : 0.951 ± 0.006
```

	eps	clusters	noise_%	ARI vs eps=0.5
0	0.40	4	18.50	0.53
1	0.45	4	12.74	0.89
2	0.50	4	8.47	1.00
3	0.55	3	6.44	0.95
4	0.60	2	4.50	0.14

A5. Tenure and K-Means versus DBSCAN cross-tabulation.

	mean	median
KMeans		
0	306.40	339.00
1	252.40	269.00
2	205.70	218.00
3	126.90	71.00

DBSCAN noise = 8.5% of customers but 46.0% of revenue

DBSCAN	-1	0	1	2	3	All
KMeans						
0	154	660	0	0	7	821
1	60	1100	85	0	0	1245
2	59	269	1111	9	0	1448
3	94	490	236	0	0	820
All	367	2519	1432	9	7	4334